# Layer Looping

Per-token layer programs in a frozen LLM:
how much is there to gain, and can the previous token see it coming?

Tyler Gee
Kāula

Preprint draft · October 29, 2026

**Abstract**

A pretrained transformer sends every token through every layer exactly once. We ask how much next-token loss is left on the table by that choice, and whether the right per-token change can be chosen *before* the token is processed. For every token of a frozen Gemma-2-2B we compute the exact change in its next-token negative log-likelihood under 104 layer programs: repeat one layer $k$ times, skip one layer, swap two adjacent layers, or loop the block of layers 10–13. Only that token's computation changes. From this per-token oracle we measure (i) headroom by edit type and layer; (ii) a signed-cost compute–accuracy frontier on which skips can pay for loops; (iii) how much of that headroom a linear router recovers from the *previous* token's internals, against a same-token router and training-free entropy- and norm-gated baselines from prior work; and (iv) whether interpretable signals (logit-lens entropy and agreement, residual update norms, attention entropy, Gemma Scope SAE latents) carry that information as well as raw hidden states. We also measure how much of the per-token gain survives when routes are applied to whole sequences, where edited tokens change what later tokens attend to.

## 1 Introduction

Adaptive-depth methods—adaptive computation time [8], universal transformers [4], early exiting [18], mixture-of-depths [17], layer dropping [5, 15]—assume tokens differ in how much computation they need. Frozen models tolerate having layers skipped, repeated, or reordered [10, 19], and recent training-free methods re-apply layers per token at high-entropy positions [2] or where the hidden-state norm peaks [21]. Static mid-block loops in frozen models give gains within noise [3, 14], and learned routers have repeatedly recovered little of oracle headroom [1, 7].

What is missing is a clean measurement: for each token in a frozen model, *exactly* how much would a small edit to its own layer schedule help, and is that predictable from information available before the token's forward pass? Our contributions:

1. **An exact per-token oracle** over repeat, skip, swap and block-loop programs in Gemma-2-2B, computed for all positions in about one forward pass per program (Section 2).
2. **A signed-cost frontier** in which skips ($-1$) pay for repeats, against static programs and training-free baselines from prior work.
3. **Routing from token $t-1$**, decided before token $t$'s pass, compared with a same-token router.
4. **A controlled comparison** of interpretable signals, raw hidden states and SAE latents as router inputs.

5. **The interaction gap**: how much of the summed per-token gain survives when whole routes are applied to sequences.

## 2 Method

**Layer programs.** Gemma-2-2B has $L = 26$ layers, applied in order $0, \ldots, L-1$. A program $c$ is one edit to this schedule for a single token: repeat@$\ell \times k$ ($k \in \{2, 3\}$, cost $k-1$), skip@$\ell$ (cost $-1$), swap@$\ell$ (run $\ell+1$ before $\ell$, cost 0), plus the static block loop of layers 10–13 run twice, the best block reported by Lys et al. [14] (cost 4). That is 104 programs. Cost counts extra layer applications.

**The diagonal swap.** We want token $t$'s next-token loss when only $t$ runs program $c$ and every other token runs normally. Because the model is causal, we can compute this for all positions at once. At each step of the program, a query attends to the *base-pass* keys and values of every other position at the layer being run, and to its *own* modified key and value on the diagonal. This is exact for each token's own loss, at the cost of about one forward pass per program. It ignores the effect of $t$'s modified key and value on later tokens; Section 3.5 measures that separately. CPU tests on tiny random Gemma-2 models check that every oracle delta equals an independent recompute that truncates the sequence at $t$ and runs the program normally.

**Oracle and routers.** $\Delta\mathrm{NLL}_t(c) = \mathrm{NLL}_t(c) - \mathrm{NLL}_t(\text{base})$; negative means the edit helps. The oracle picks $\arg\min_c \Delta\mathrm{NLL}_t(c)$, falling back to the base program when nothing helps. A token is *helped* if the oracle gain exceeds 0.05 nats, and a skip is *safe* if its $\Delta\mathrm{NLL} < 0.01$. A router predicts the vector $\widehat{\Delta\mathrm{NLL}}_t$ by ridge regression and routes to $\arg\min_c [\widehat{\Delta\mathrm{NLL}}_t(c) + \lambda\,\mathrm{cost}(c)]$ (base if the minimum is non-negative); sweeping $\lambda$ traces its frontier, and we always report *realized* $\Delta$NLL. The **same-token** router reads token $t$'s base-pass features; the **lagged** router reads token $t-1$'s, which exist before $t$ is processed.

**Features and baselines.** Interpretable features are four per-layer groups: logit-lens [16] entropy, logit-lens agreement with the final prediction, relative residual-update norm, and attention entropy. We compare them with raw hidden states and with Gemma Scope SAE [13] latents at layers 6, 12 and 18. Training-free baselines: an EGLR-like rule [2] that repeats a top layer when next-token entropy is high (decided at $t$, or one token early), and a norm-peak rule [21] that repeats the layer with the largest update norm.

**Data and evaluation.** WikiText-103 prose and GSM8K question–answer text in alternating 512-token chunks, each starting with BOS. Chunks with index $\equiv 0 \pmod 5$ are held out; 95% confidence intervals come from bootstrap resampling over held-out chunks.

## 3 Results

### 3.1 Headroom (E1)

Over [pending] positions with base NLL [pending] nats/token, the per-token oracle reduces NLL by [pending] nats ([pending]% of base) and helps [pending] of tokens. The best static program, applied to every token, is [pending] with mean $\Delta$NLL [pending]. A safe skip exists for [pending] of tokens.

Table 1: Oracle headroom by edit type. *Restricted gain*: oracle gain using only this edit type. *Best-op share*: fraction of all tokens that are helped and whose best edit is this type.

| Edit | Restricted gain (nats) | Best-op share | Best static |
|---|---|---|---|
| repeat / skip / swap / block | [pending] | [pending] | [pending] |

### 3.2 Compute–accuracy frontier (E2)

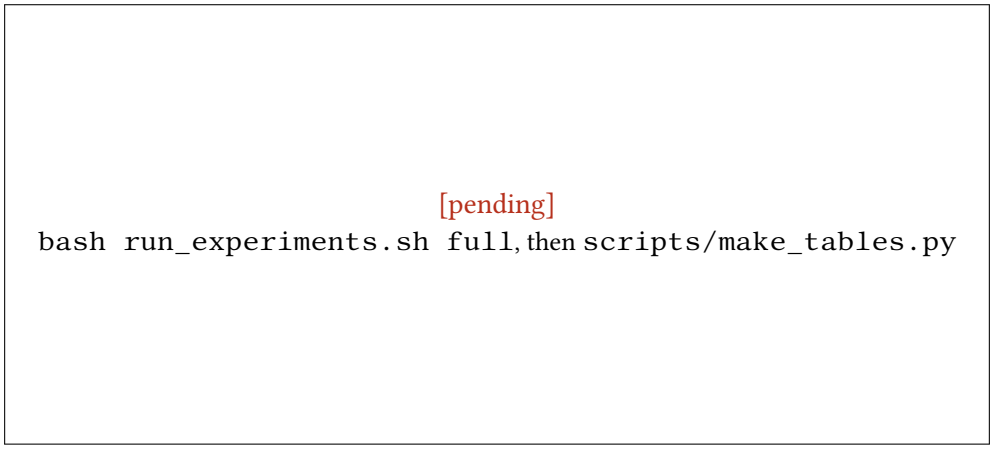



Figure 1: NLL saved versus extra layer applications per token. Curves are Pareto fronts over $\lambda$; markers are static programs.

Table 2: Budget-matched ΔNLL (nats/token; lower is better): the best frontier point using at most the given extra layer applications per token. "—": unreachable.

[pending]

### 3.3 Does the previous token know? (E3)

The lagged interpretable router retains [pending] of the same-token router's gain.

### 3.4 What carries the signal? (E4)

[pending]

### 3.5 Interaction gap (E5)

[pending] Whole-sequence routed loss versus the sum of independent per-token oracle deltas, from `routed_eval.py` with first-pass KV caching.

## 4 Related work

**Per-token computation in frozen models.** EGLR [2] re-applies the top layers at high-entropy tokens and the Tell-Tale Norm [21] recurses layers guided by hidden-state norms; both decide from

Table 3: Routers at $\lambda = 0$ on held-out chunks. AUC: detecting helped tokens from the predicted best gain. *Edit%*: share of tokens routed away from the base program.

| Features | ΔNLL | 95% CI | AUC | Edit% / +layers |
|---|---|---|---|---|
| [pending] | | | | |

Table 4: Lagged routers using one interpretable feature group ($\lambda = 0$).

| Feature group | ΔNLL | +layers |
|---|---|---|
| [pending] | | |

the current token, after its pass. Think-at-Hard [6] adds whole-model iterations for hard tokens with a trained backbone. **Programs of layers.** CoLa [11] searches per-sample skip/repeat chains, POLAR [12] learns programs of layers, and Dr.LLM [9] trains sequence-level routers. **Static loops.** Inner Loop Inference [14] and Training-Free Looped Transformers [3] loop fixed blocks. **Routers versus oracles.** Glavas et al. [7] find per-token skip controllers no better than a constant input, and Duo-LLM [1] finds large gaps between oracles and learned routers. Wei et al. [20] argue skip headroom is shrinking in modern models. **Adaptive depth with training.** ACT [8], universal transformers [4], CALM [18], mixture-of-depths [17].

## 5 Limitations

The oracle is a hindsight upper bound: it picks the best of 104 outcomes per token, which rewards any edit that perturbs a wrong prediction. Per-token deltas ignore effects on later tokens; E5 measures that gap. Routers are linear, the model is a single 2B checkpoint, and loss is measured with teacher forcing.

## References


[1] Keivan Alizadeh, Iman Mirzadeh, Hooman Shahrokhi, Dmitry Belenko, Frank Sun, Minsik Cho, et al. Duo-LLM: A framework for studying adaptive computation in large language models. *arXiv preprint arXiv:2410.10846*, 2024.

[2] Soham Bhattacharjee, Dushyant Singh Chauhan, Salem Lahlou, Martin Takac, and Nils Lukas. Entropy-gated latent recursion. *arXiv preprint arXiv:2606.16620*, 2026.

[3] Lizhang Chen, Jonathan Li, Chen Liang, Ni Lao, and Qiang Liu. Training-free looped transformers. *arXiv preprint arXiv:2605.23872*, 2026.

[4] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

[5] Angela Fan, Edouard Grave, and Armand Joulin. Reducing transformer depth on demand with structured dropout. In *International Conference on Learning Representations*, 2020.

[6] Tianyu Fu, Yichen You, Zekai Chen, Guohao Dai, Huazhong Yang, Yu Wang, et al. Think-at-hard: Dynamic looped transformers for improved reasoning. *arXiv preprint arXiv:2511.08577*, 2025.

[7] Theodore Glavas, Joud Chataoui, Florence Regol, Wassim Jabbour, Antonios Valkanas, Boris N. Oreshkin, et al. Dynamic layer selection in decoder-only transformers. *arXiv preprint arXiv:2410.20022*, 2024.

[8] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[9] Ahmed Heakl, Martin Gubri, Salman Khan, Sangdoo Yun, and Seong Joon Oh. Dr.LLM: Dynamic layer routing in LLMs. *arXiv preprint arXiv:2510.12773*, 2025.

[10] Vedang Lad, Jin Hwa Lee, Wes Gurnee, and Max Tegmark. The remarkable robustness of LLMs: Stages of inference? *arXiv preprint arXiv:2406.19384*, 2024.

[11] Ziyue Li, Yang Li, and Tianyi Zhou. Skip a layer or loop it? test-time depth adaptation of pretrained LLMs. *arXiv preprint arXiv:2507.07996*, 2025.

[12] Ziyue Li, Yang Li, and Tianyi Zhou. Skip a layer or loop it? learning program-of-layers in LLMs. *arXiv preprint arXiv:2606.06574*, 2026.

[13] Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma scope: Open sparse autoencoders everywhere all at once on Gemma 2. *arXiv preprint arXiv:2408.05147*, 2024.

[14] Jonathan Lys, Vincent Gripon, Bastien Pasdeloup, Axel Marmoret, Lukas Mauch, Fabien Cardinaux, et al. Inner loop inference for pretrained transformers: Unlocking latent capabilities without training. *arXiv preprint arXiv:2602.14759*, 2026.

[15] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. ShortGPT: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[16] nostalgebraist. interpreting GPT: the logit lens. LessWrong, 2020. URL https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens.

[17] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[18] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*, 2022.

[19] Qi Sun, Marc Pickett, Aakash Kumar Nain, and Llion Jones. Transformer layers as painters. *arXiv preprint arXiv:2407.09298*, 2024.

[20] Rui Wei, Rui Du, Hanfei Yu, Devesh Tiwari, Jian Li, Zhaozhuo Xu, et al. The diminishing returns of early-exit decoding in modern LLMs. *arXiv preprint arXiv:2603.23701*, 2026.

[21] Jinyang Zhang, Hongxin Ding, Yue Fang, Weibin Liao, Muyang Ye, Junfeng Zhao, et al. The tell-tale norm: $\ell_2$ magnitude as a signal for reasoning dynamics in large language models. *arXiv preprint arXiv:2606.06188*, 2026.